

# MOLLY MARKET

SYSTÈME INTÉGRÉ DE GESTION DE SUPERMARCHÉ & POINT DE VENTE (POS)

## MASTER DOSSIER TECHNIQUE D'ARCHITECTURE

& GUIDE EXPLICATIF FICHIER PAR FICHIER POUR LA SOUTENANCE

### CONTENU DU DOSSIER & GUIDE DE DÉMONSTRATION :

- PARTIE 1 : Base de Données SQL (01 à 10) - Schéma, Triggers, Procédures, Rôles & PUMP
- PARTIE 2 : Backend API Express (server/index.js, db.js, auth.js) - Délégation 100% SQL
- PARTIE 3 : Frontend React 19 (POS, Caisse, Achats, IndexedDB, Impression ESC/POS)
- PARTIE 4 : Guide Chronologique de Soutenance Orale (Discours & Timing minute par minute)
- PARTIE 5 : 5 Scénarios de Démonstration en Direct (Miroir SQL & Interface Utilisateur)
- PARTIE 6 : Fiches Réponses aux Questions Pièges & FAQ Technique du Jury
- Conformité Comptable : Francs CFA (XOF / FCFA) & Système Comptable OHADA

### PROPRIÉTÉ & CONTEXTE DU PROJET

Supermarché Molly Market Abidjan • République de Côte d'Ivoire

Architecture 3-Tiers : React 19 + Node.js 22 LTS + PostgreSQL 16+ PL/pgSQL

1. VUE D'ENSEMBLE DE L'ARCHITECTURE 3-TIERS

Molly Market repose sur le principe fondamental de la "Database-First Business Logic" : aucun calcul critique (TVA, validation de stock, valorisation d'inventaire, ventilation de caisse, PUMP) n'est confié arbitrairement au client JavaScript ou à Node.js. PostgreSQL est le garant unique de l'atomicité ACID et de la cohérence comptable.

| Couche / Module                           | Technologies & Composants                                               | Rôle Métier & Valeur Ajoutée                                                                                                                                                 |
|-------------------------------------------|-------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Couche Présentation (Frontend)            | React 19, TypeScript 5.8, Vite 6.2, Tailwind CSS, Zustand, Lucide React | Interface caisse POS réactive, scan codes-barres, réconciliation de caisse, tableaux de bord de gestion et impression thermique.                                             |
| Résilience Hors-Ligne (Offline-First)     | API IndexedDB (mollymarket_pos_offline), Hook useOnlineStatus           | Permet l'encaissement continu en cas de coupure de réseau. Les ventes en attente sont stockées localement et synchronisées automatiquement dès le retour du serveur.         |
| Couche Service (Backend REST)             | Node.js 22 LTS, Express 4.21, pg.Pool (node-postgres), JSON Web Token   | Passerelle HTTP stateless ultra-légère. Elle vérifie les jetons JWT, applique le contrôle RBAC et délègue 100% des opérations aux procédures stockées SQL via CALL.          |
| Couche Données & Métier (Base de Données) | PostgreSQL 16+, PL/pgSQL, Triggers, Vues Métier, Index B-Tree, RLS      | Moteur décisionnel et transactionnel. Exécute les procédures stockées (CALL), garantit l'intégrité des stocks par triggers et maintient l'historique immuable (Audit Trail). |
| Périphériques Caisse                      | Protocole ESC/POS 80mm/58mm, WebSerial / WebUSB / Impression brute      | Génération des flux binaires d'impression pour tickets thermiques de supermarché avec ouverture automatique du tiroir-caisse et coupe-papier.                                |
| Plan de Sauvegarde                        | Scripts batch & Node.js avec pg_dump / pg_restore et rétention 7j       | Sauvegardes journalières horodatées, dumps binaires et SQL clairs avec rotation automatique.                                                                                 |

2. FLUX TRANSACTIONNEL DÉTAILLÉ D'UNE VENTE AU SUPERMARCHÉ

|                             |                                                                                                           |
|-----------------------------|-----------------------------------------------------------------------------------------------------------|
| 1. Bip Code-Barres          | Le lecteur scanne l'article (EAN-13). Le frontend interroge le catalogue mémoire et incrémente le panier. |
| 2. Choix Paiement           | Le caissier sélectionne le mode : Espèces, Wave Mobile Money, Orange Money, MTN ou Carte.                 |
| 3. Requête API Sécurisée    | POST /api/ventes avec Bearer Token JWT contenant le client_id, le vendeur_id et le panier JSON.           |
| 4. Exécution PL/pgSQL       | Le backend exécute CALL effectuer_vente(...) -> Début de transaction atomique PostgreSQL.                 |
| 5. Déclenchement Trigger    | L'insertion dans lignes_vente déclenche trg_decrements_stock_vente qui décrémente stock_actuel.           |
| 6. Verrouillage Concurrency | Si stock_actuel < quantite, le trigger lève une exception SQL qui ROLLBACK l'intégralité de la vente.     |
| 7. Ticket & Impression      | PostgreSQL retourne le numéro officiel TK-YYYYMMDD-XXX et le ticket thermique 80mm est généré.            |

1. FICHER 01\_tables.sql : SCHÉMA RELATIONNEL & CONTRAINTES

Comment expliquer ce fichier au jury : "Ce script crée l'intégralité du modèle relationnel en respectant la 3ème Forme Normale (3NF). Il définit 11 tables relationnelles, 6 types énumérés ENUM, des contraintes d'intégrité CHECK (prix >= 0, stock >= 0), des clés étrangères avec ON DELETE RESTRICT et un plan d'index B-Tree."

| Table SQL              | Colonnes Clés                                                                          | Contraintes & Explication Soutenance                                                     |
|------------------------|----------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|
| utilisateurs           | id, matricule, nom, prenom, email, mot_de_passe, role, telephone, actif                | CHECK sur format email et matricule unique. Gère les comptes du personnel.               |
| caisses                | id, code, nom, emplacement, statut, derniere_activite                                  | Terminaux physiques de vente (Caisse 1 Centrale, Caisse 2 Express).                      |
| categories             | id, nom, description, icone, couleur, actif                                            | Rayons du supermarché (Alimentation, Boissons, Entretien, Vêtements, Cosmétiques).       |
| produits               | id, code_barre, nom, categorie_id, prix_vente, prix_achat, stock_actuel, seuil_alerte  | CHECK (prix_vente >= 0 AND prix_achat >= 0), EAN-13 UNIQUE, Index B-Tree sur code_barre. |
| ventes                 | id, numero_ticket, client_id, vendeur_id, caisse_id, date_vente, montant_total, statut | En-tête des tickets TK-YYYYMMDD-XXX, liaison caisse_id pour gestion multi-caisses.       |
| lignes_vente           | id, vente_id, produit_id, quantite, prix_unitaire, montant_total                       | Lignes du ticket. Le trigger décrémente stock_actuel et recalcule ventes.montant_total.  |
| paiements              | id, reference_paiement, vente_id, montant, mode_paiement, date_paiement, statut        | Ventilation financière : especes, wave, orange_money, mtn_money, carte, cheque.          |
| points_caisse          | id, numero_session, caisse_id, date_journee, heure_ouverture, statut, fond_initial     | Sessions de caisse (statuts : ouverte, soumise_directeur, validee_directeur).            |
| billetage_point_caisse | id, point_caisse_id, mode_paiement, montant_theorique, montant_compte, ecart           | Réconciliation et calcul automatique des écarts de caisse en fin de journée.             |
| mouvements_stock       | id, produit_id, type_mouvement, quantite, stock_avant, stock_apres, motif              | Audit Trail immuable de chaque variation (vente, achat, inventaire, casse, retour).      |
| fournisseurs / achats  | id, code_fournisseur, nom_entreprise / id, numero_facture, statut, montant             | Circuit d'approvisionnement : commande -> réception magasinier -> paiement directeur.    |

2. FICHER 02\_roles\_privileges.sql : MODÈLE DE SÉCURITÉ RBAC & RLS

| Rôle PostgreSQL    | Privilèges SQL (GRANT / REVOKE)                                                  | Explication Rôle Métier                                                                               |
|--------------------|----------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| administrateur     | ALL PRIVILEGES sur toutes les tables, séquences et procédures.                   | Superviseur technique : gestion des utilisateurs, sauvegardes, restauration et maintenance.           |
| directeur          | SELECT sur toutes les vues, EXECUTE sur validation de caisse et paiement achats. | Direction commerciale : consultation des statistiques, validation finale des caisses et décaissement. |
| vendeur (caissier) | SELECT sur produits/clients, INSERT sur ventes/lignes_vente/paiements.           | Opérateur caisse : encaissement POS, saisie du billetage. Aucune modification de prix autorisée.      |
| magasinier         | SELECT/UPDATE sur produits, INSERT sur réceptions achats & ajustements.          | Gestionnaire de stock : réception des livraisons fournisseurs, inventaires et alertes ruptures.       |

1. FICHER 03\_vues.sql : VUES DE REPORTING & JOINTURES COMPLEXE

Comment expliquer ce fichier au jury : "Les vues masquent la complexité des jointures relationnelles et offrent des flux pré-agrégés sécurisés pour les tableaux de bord du Directeur et du Magasinier sans duplication."

| Nom de la Vue    | Structure & Définition SQL                                                                                                                              | Rôle Métier                                                                                           |
|------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| vue_stock        | SELECT p.*, c.nom AS categorie, CASE WHEN stock_actuel <= seuil_alerte THEN 'critique' ELSE 'ok' END AS etat_stock FROM produits p JOIN categories c... | Fournit au magasinier l'état consolidé du stock avec badge d'alerte rupture automatique.              |
| vue_commandes    | SELECT v.*, u.nom AS vendeur_nom, cl.nom AS client_nom, p.mode_paiement, p.statut AS statut_paiement FROM ventes v JOIN utilisateurs u...               | Expose l'historique complet des tickets de caisse avec le vendeur, le client et le mode de règlement. |
| vue_achats       | SELECT a.*, f.nom_entreprise AS fournisseur_nom, u.nom AS acheteur_nom FROM achats a JOIN fournisseurs f...                                             | Consolide le circuit d'approvisionnement : statut de livraison, montant total et état de paiement.    |
| vue_statistiques | Agrégations GROUP BY : CA journalier, CA mensuel, panier moyen, nombre de tickets et répartition par mode.                                              | Alimente les graphiques du tableau de bord de la direction en temps réel.                             |
| vue_top_clients  | SELECT cl.*, COUNT(v.id) AS nb_achats, SUM(v.montant_total) AS total_depense FROM clients cl JOIN ventes v...                                           | Identifie les meilleurs clients pour la politique de fidélité du supermarché.                         |

2. FICHER 04\_fonctions.sql : FONCTIONS MÉTIER PL/pgSQL

| Fonction PL/pgSQL                       | Type Retour        | Logique Exécutée & Explication Soutenance                                                                                                               |
|-----------------------------------------|--------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| fn_authentifier_utilisateur(email, mdp) | RETURNS TABLE(...) | Vérifie l'email et compare le hash du mot de passe avec pgcrypto (crypt / gen_salt). Retourne le profil et le rôle si valide, lève une exception sinon. |
| chiffre_affaires(debut, fin)            | RETURNS NUMERIC    | Calcule le chiffre d'affaires net encaissé sur une plage de dates en ne retenant que les ventes au statut 'terminee'.                                   |
| stock_disponible(produit_id)            | RETURNS INTEGER    | Retourne la quantité réelle disponible en rayon après déduction des réservations en cours.                                                              |
| fn_generer_numero_ticket()              | RETURNS VARCHAR    | Génère une référence unique TK-YYYYMMDD-XXXX basée sur la séquence du jour.                                                                             |
| fn_generer_reference_paiement()         | RETURNS VARCHAR    | Génère la référence comptable PAY-YYYYMMDD-XXXX pour la traçabilité fiduciaire.                                                                         |

### 1. DÉCORTICAGE LIGNE PAR LIGNE DE LA PROCÉDURE effectuer\_vente

C'est la procédure la plus importante de la soutenance. Expliquez au jury comment chaque ligne garantit l'atomicité : "La procédure prend en entrée le panier JSON du client. Si un seul article manque, tout est annulé."

| Bloc / Lignes  | Étape Procédure             | Explication Précise du Code PL/pgSQL                                                                                                                        |
|----------------|-----------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Lignes 140-148 | En-tête & Paramètres        | CREATE PROCEDURE effectuer_vente(p_client_id, p_lignes JSON, p_vendeur_id, p_mode_paiement, INOUT p_vente_id, INOUT p_numero_ticket, INOUT p_montant_total) |
| Lignes 149-155 | Génération Ticket & En-tête | v_numero_ticket := fn_generer_numero_ticket(); -> INSERT INTO ventes (...) RETURNING id INTO p_vente_id; Initialise la vente à 0 FCFA.                      |
| Lignes 156-160 | Boucle sur le Panier JSON   | FOR v_ligne IN SELECT * FROM json_array_elements(p_lignes) LOOP -> Itère sur chaque article envoyé par le frontend.                                         |
| Lignes 161-172 | Vérification Stock & Verrou | SELECT stock_actuel, prix_vente INTO v_produit FROM produits WHERE id = ...; IF stock_actuel < quantite THEN RAISE EXCEPTION 'Stock insuffisant...';        |
| Lignes 173-181 | Insertion Ligne & Trigger   | INSERT INTO lignes_vente(vente_id, produit_id, quantite, prix_unitaire) VALUES (...); -> Déclenche automatiquement le trigger de stock.                     |
| Lignes 182-185 | Recalcul du Montant Total   | SELECT montant_total INTO p_montant_total FROM ventes WHERE id = p_vente_id; Récupère le total exact recalculé par le trigger.                              |
| Lignes 186-191 | Paiement & Clôture          | INSERT INTO paiements(reference_paiement, vente_id, montant, mode_paiement, statut) VALUES (fn_generer_ref(), p_vente_id, p_montant_total, ...);            |

### 2. AUTRES PROCÉDURES STOCKÉES STRATÉGIQUES DU SYSTÈME

| Nom Procédure                                           | Logique de Fonctionnement & Sécurité                                                                                                                                                     |
|---------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| annuler_vente(vente_id, motif, user_id)                 | Vérifie que la vente est 'terminee'. Boucle sur les lignes_vente, incrémente le stock_actuel, crée une entrée mouvement_stock de type 'annulation_vente' et passe le statut à 'annulee'. |
| effectuer_achat(fournisseur_id, lignes, user_id)        | Crée une commande d'achat fournisseur en statut 'en_attente'. Insère les lignes d'achat avec les quantités et prix d'achat négociés.                                                     |
| reception_stock(achat_id, user_id)                      | Exécutée par le Magasinier. Incrémente le stock des articles reçus, recalcule le Prix Unitaire Moyen Pondéré (PUMP) et passe l'achat à 'receptionne'.                                    |
| payer_facture_fournisseur(achat_id, mode, user_id)      | Exécutée par le Directeur. Valide le règlement de la facture fournisseur, génère un décaissement et met à jour la trésorerie de caisse.                                                  |
| ajuster_stock(produit_id, nouvelle_qte, motif, user_id) | Ajustement après inventaire physique. Calcule le delta, met à jour le stock et journalise la justification dans mouvements_stock.                                                        |

1. FICHIER 06\_triggers.sql : DÉCLENCHEURS & FORMULE DU PUMP

| Déclencheur / Règle                  | Cible & Événement                   | Action & Règle Métier                                                                                                             |
|--------------------------------------|-------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| trg_interdire_prix_negatif           | BEFORE INSERT OR UPDATE ON produits | Vérifie IF NEW.prix_vente < 0 OR NEW.prix_achat < 0 THEN RAISE EXCEPTION. Empêche toute incohérence tarifaire.                    |
| trg_empecher_stock_negatif           | BEFORE UPDATE ON produits           | Vérifie IF NEW.stock_actuel < 0 THEN RAISE EXCEPTION. Protège contre les ruptures fantômes et ventes à découvert.                 |
| trg_calcul_montant_commande          | AFTER INSERT ON lignes_vente        | Recalcule automatiquement le montant_total de la table ventes par SUM(quantite * prix_unitaire).                                  |
| trg_audit_mouvement_stock            | AFTER INSERT ON mouvements_stock    | Garantit l'immuabilité : aucune ligne de mouvement_stock ne peut être modifiée ou supprimée (Audit Trail comptable).              |
| Calcul du PUMP (Réapprovisionnement) | Formule Mathématique Standard       | $PUMP\_nouveau = \frac{[(Stock\_actuel * PUMP\_ancien) + (Qte\_achetee * Prix\_achat\_nouveau)]}{(Stock\_actuel + Qte\_achetee)}$ |

2. FICHIERS 07\_cte.sql, 08, 09 & 10 : REQUÊTES AVANCÉES & EXPLOITATION

| Fichier & Thème                               | Détails Techniques & Points Clés pour la Soutenance                                                                                                                                                                                                                                                                                   |
|-----------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 07_cte.sql (Requêtes Analytiques & Fenêtrage) | <ul style="list-style-type: none"><li>Segmentation RFM (Récence, Fréquence, Montant) avec NTILE(4) OVER (...)</li><li>Palmarès des ventes par catégorie avec DENSE_RANK() OVER (PARTITION BY categorie_id ORDER BY total DESC)</li><li>Évolution cumulative du CA avec SUM(montant_total) OVER (ORDER BY date_vente).</li></ul>       |
| 08_seed_data.sql (Jeu de Données)             | <ul style="list-style-type: none"><li>130+ références réelles (Produits de grande consommation d'Abidjan, boissons, épicerie, textile).</li><li>4 utilisateurs opérationnels avec mots de passe chiffrés (Admin, Directeur Eden, Caissier Noam, Magasinier).</li></ul>                                                                |
| 09_sauvegarde_restauraton.sql + .bat          | <ul style="list-style-type: none"><li>Sauvegarde binaire haute performance : pg_dump -Fc -d mollymarket_backend -f backup.dump</li><li>Restauration transactionnelle : pg_restore --clean --if-exists -d mollymarket_backend backup.dump</li><li>Script batch backup_mollymarket.bat avec rotation automatique sur 7 jours.</li></ul> |
| 10_import_export_copy.sql (Import CSV)        | <ul style="list-style-type: none"><li>Import massif haute vitesse via la commande SQL : \copy produits FROM 'sql/data/produits.csv' WITH (FORMAT csv, HEADER true, DELIMITER ',')</li><li>Idéal pour initialiser 50 000 articles en moins d'une seconde.</li></ul>                                                                    |

1. STRUCTURE DU BACKEND & DÉLÉGATION 100% SQL

Comment expliquer le backend au jury : "Le backend Node.js / Express a été conçu comme une passerelle HTTP stateless sécurisée. Son rôle se limite à l'authentification JWT, la validation des paramètres et l'appel direct des procédures stockées PostgreSQL (CALL) et des vues métier. Aucune logique métier n'est codée en dur en JavaScript."

| Fichier Backend                                   | Description Ligne par Ligne & Rôle                                                                                                                                                                                                                                                                                                                                                                                      |
|---------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| server/db.js (Gestion de la Connexion Pool)       | Lignes 1-35 : Instanciation du pool `pg.Pool` avec configuration des variables d'environnement (.env). Gère la réutilisation des connexions TCP, élimine les fuites de mémoire et expose la méthode `query(sql, params)` avec support des transactions et gestion des erreurs de déconnexion inattendue.                                                                                                                |
| server/middleware/auth.js (JWT & Contrôle RBAC)   | Lignes 1-45 : Middleware `verifierToken` qui extrait le header `Authorization: Bearer <token>` et valide la signature avec `jwt.verify(token, JWT_SECRET)`. En cas de succès, injecte `req.utilisateur = decoded`. Middleware `exigerRole(...roles)` qui vérifie si le rôle de l'utilisateur a le droit d'accéder à la route (ex: Directeur/Admin).                                                                     |
| server/index.js (Routes REST & Appels Procédures) | Lignes 1-80 : Initialisation Express, CORS, parseur JSON et montage des middlewares.<br>Lignes 81-220 : Routes d'authentification (`POST /api/auth/login` appelant `fn_authentifier_utilisateur`).<br>Lignes 221-450 : Routes Caisse & Ventes (`POST /api/ventes` appelant `CALL effectuer_vente(\$1, \$2, ...)`).<br>Lignes 451-650 : Routes Point de Caisse, Trésorerie (`/api/caisse/solde`) et Achats Fournisseurs. |

2. CARTOGRAPHIE DES ROUTES API EXPRESS <-> OBJETS POSTGRESQL

| Route API REST                    | Accès RBAC        | Objet SQL PostgreSQL Appelé                               | Rôle Métier                                            |
|-----------------------------------|-------------------|-----------------------------------------------------------|--------------------------------------------------------|
| POST /api/auth/login              | Public            | SELECT * FROM fn_authentifier_utilisateur(\$1, \$2)       | Authentification sécurisée avec hash pgcrypto.         |
| POST /api/ventes                  | Vendeur / Dir     | CALL effectuer_vente(\$1, \$2, \$3, \$4, \$5, \$6, \$7)   | Création ticket, boucle JSON, trigger stock, paiement. |
| POST /api/ventes/:id/annuler      | Directeur / Admin | CALL annuler_vente(\$1, \$2, \$3)                         | Annulation transactionnelle et retour en stock.        |
| GET /api/caisse/solde             | Tous connectés    | Calcul agrégé : Fond + Ventes Espèces + Entrées - Sorties | Solde de trésorerie physique en temps réel.            |
| POST /api/achats                  | Magasinier / Dir  | CALL effectuer_achat(\$1, \$2, \$3)                       | Création d'une commande fournisseur.                   |
| POST /api/achats/:id/receptionner | Magasinier / Dir  | CALL reception_stock(\$1, \$2)                            | Entrée de stock et mise à jour PUMP.                   |
| POST /api/achats/:id/payer        | Directeur         | CALL payer_facture_fournisseur(\$1, \$2, \$3)             | Paiement fournisseur et sortie de caisse automatique.  |

## 1. DÉCORTICAGE DES COMPOSANTS FRONTEND MAJEURS

Comment expliquer le frontend au jury : "L'interface est développée en React 19 et TypeScript avec Vite. Elle combine une ergonomie fluide pour le passage en caisse rapide et des outils de contrôle stricts pour la direction."

| Fichier Composant              | Module Métier                   | Fonctionnalités & Points Forts                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|--------------------------------|---------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| SupermarchePOSTerminal.tsx     | Écran de Vente & Caisse POS     | <ul style="list-style-type: none"><li>• Écouteur global d'événements clavier pour douchette code-barres (capture USB standard).</li><li>• Panier interactif : calcul immédiat TTC/HT, gestion des quantités et remises autorisées.</li><li>• Modal Multi-Moyens : choix Espèces (avec calcul de monnaie à rendre), Wave, Orange Money, Carte.</li><li>• Intégration de la file IndexedDB pour continuité hors-ligne et bouton d'impression ESC/POS.</li></ul>                   |
| PointDeCaissePage.tsx          | Clôture & Réconciliation Caisse | <ul style="list-style-type: none"><li>• Affichage des montants théoriques calculés par la base de données pour chaque moyen de règlement.</li><li>• Grille de saisie du comptage physique réel (billets de 10 000, 5 000, 2 000, 1 000, 500 et pièces).</li><li>• Calcul automatique de l'écart en temps réel (Vert = Conforme, Rouge = Écart).</li><li>• Circuit de double validation : soumission par le Caissier -&gt; validation finale et visa par le Directeur.</li></ul> |
| AchatsPage.tsx                 | Approvisionnements Fournisseurs | <ul style="list-style-type: none"><li>• Création de bons de commande avec sélection des fournisseurs d'Abidjan et lignes d'articles.</li><li>• Écran de réception magasinier avec vérification des quantités livrées.</li><li>• Module de paiement directeur : décaissement espèces ou virement avec débit automatique du solde de caisse.</li></ul>                                                                                                                            |
| DirecteurDashboard.tsx / Admin | Tableaux de Bord & KPIs         | <ul style="list-style-type: none"><li>• Graphiques de tendance du CA journalier/mensuel (Recharts), palmarès des meilleures ventes.</li><li>• Alertes ruptures de stocks, audit des mouvements et panneau de gestion des utilisateurs.</li></ul>                                                                                                                                                                                                                                |

## 2. SERVICES TECHNIQUES : OFFLINE-FIRST & IMPRESSION THERMIQUE ESC/POS

| Composant Technique                 | Description & Rôle dans le Système                                                                                                                                             |
|-------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| src/services/api.ts                 | Client Axios centralisé avec intercepteur JWT. Injecte automatiquement le token dans Authorization: Bearer et intercepte les erreurs 401 pour redirection sécurisée.           |
| src/hooks/useOnlineStatus.ts        | Surveille l'état réseau via window.addEventListener('online'/'offline') et effectue des pings légers vers le backend pour basculer instantanément en mode dégradé.             |
| IndexedDB (mollymarket_pos_offline) | Base de données locale NoSQL du navigateur. Stocke les ventes en attente (pending_sales) et émet un ticket provisoire TK-OFF-XXXXXX sans bloquer le caissier.                  |
| src/utills/escpos.ts                | Générateur de commandes binaires pour imprimantes thermiques 80mm/58mm (Epson TM-T20, Xprinter) : initialisation ESC @, alignement ESC a, texte gras ESC E, coupe papier GS V. |



1. DÉROULEMENT MINUTE PAR MINUTE DE VOTRE SOUTENANCE

| Timing                   | Phase de Présentation                    | Discours & Actions Recommandées Devant le Jury                                                                                                                                                                                                                                                                                                                                                             |
|--------------------------|------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0:00 - 2:00<br>(2 min)   | Introduction & Contexte Métier           | <ul style="list-style-type: none"><li>• Saluer le jury et introduire le projet Molly Market : supermarché moderne à Abidjan.</li><li>• Poser la problématique : rapidité en caisse, fiabilité des stocks, multi-paiements (Wave, Espèces), et continuité en cas de coupure internet.</li><li>• Annoncer la démarche : architecture Database-First garantissant une intégrité absolue.</li></ul>            |
| 2:00 - 5:30<br>(3.5 min) | Démonstration POS & Encaissement         | <ul style="list-style-type: none"><li>• Se connecter en Caissier (Noam Koffi) sur le terminal Caisse Centrale.</li><li>• Bip rapide de 2 articles au code-barres (Lait Bonnet Rouge + Riz Dinor).</li><li>• Choisir le paiement Espèces avec rendu de monnaie ou Wave Mobile Money.</li><li>• Valider : montrer la génération du ticket officiel TK-20260905-001 et l'impression thermique 80mm.</li></ul> |
| 5:30 - 9:00<br>(3.5 min) | Zoom Moteur SQL, Triggers & Procédures   | <ul style="list-style-type: none"><li>• Ouvrir le code SQL : montrer la procédure stockée `effectuer_vente` et la boucle JSON.</li><li>• Expliquer le trigger `trg_decrementer_stock_vente` et le verrou anti-stock négatif.</li><li>• Montrer la vue `vue_stock` et le recalcul automatique du PUMP lors des réceptions fournisseurs.</li></ul>                                                           |
| 9:00 - 12:00<br>(3 min)  | Clôture Caisse, Hors-Ligne & Sauvegardes | <ul style="list-style-type: none"><li>• Aller sur le Point de Caisse : montrer le calcul théorique, saisir le billettage physique (Écart = 0 FCFA).</li><li>• Soumettre au Directeur -&gt; Se connecter en Directeur Eden Touré pour valider et tirer le PV.</li><li>• Démontrer la résilience hors-ligne avec IndexedDB et lancer le script de sauvegarde `backup.bat`.</li></ul>                         |
| 12:00 - 15:00<br>(3 min) | Conclusion & Réponses aux Questions      | <ul style="list-style-type: none"><li>• Synthétiser les points forts : Performance (&lt; 10ms), Sécurité RBAC/JWT, Zéro calcul arbitraire JS.</li><li>• Remercier le jury et ouvrir la session de questions/réponses avec assurance.</li></ul>                                                                                                                                                             |

2. PHRASES CLÉS & VOCABULAIRE TECHNIQUE VALORISANT

|                          |                                                                                                                                                                                                      |
|--------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Sur la Base de Données : | "Nous avons fait le choix stratégique de sceller les règles de gestion au niveau du SGBD PostgreSQL sous forme de procédures stockées PL/pgSQL pour garantir le respect strict des propriétés ACID." |
| Sur le Mode Hors-Ligne : | "Grâce au stockage NoSQL local IndexedDB, une rupture de connectivité n'interrompt jamais l'activité commerciale du supermarché ; les transactions sont rejouées atomiquement dès la reconnexion."   |
| Sur la Sécurité RBAC :   | "La sécurité est appliquée en profondeur : jeton JWT signé côté API et politiques de rôles granulaires au niveau de PostgreSQL interdisant toute modification non autorisée."                        |

SCÉNARIOS DE DÉMONSTRATION DIRECTE DEVANT LE JURY (MIROIR SQL & UI)

| Scénario Métier                                                        | Protocole de Démonstration (Interface Web & Preuve SQL)                                                                                                                                                                                                                                                                                         |
|------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Scénario A : Vente Multi-Moyens & Décrémentation Stock                 | Interface : Dans le POS, scanner 2x "Huile Dinor 1L" (Code: 6181100380021). Choisir paiement "Wave Mobile Money". Cliquer sur Valider.<br>Preuve SQL : Exécuter `SELECT * FROM vue_stock WHERE id = 2;` -> Constaté la diminution exacte de 2 unités et l'insertion dans `mouvements_stock`.                                                    |
| Scénario B : Commande Fournisseur -> Réception PUMP -> Paiement Caisse | Interface : 1. Magasinier crée un achat de 50 packs d'eau Awa. 2. Clique sur "Réceptionner le stock". 3. Directeur va dans "Achats" et clique sur "Payer la facture (Espèces)".<br>Preuve SQL : Vérifier `SELECT stock_actuel, prix_achat FROM produits WHERE id=1;` (PUMP recalculé) et `SELECT solde_actuel FROM vue_solde_caisse;` (Débité). |
| Scénario C : Clôture de Caisse & Réconciliation Billetage              | Interface : Aller sur "Point de Caisse". Saisir le comptage des billets dans le tiroir. L'écart s'affiche à 0 FCFA. Cliquer sur "Soumettre au Directeur".<br>Preuve SQL : `SELECT * FROM billetage_point_caisse WHERE point_caisse_id = ...;` -> Écart = 0.00 FCFA. Session verrouillée.                                                        |
| Scénario D : Test de Rupture & Rejet Atomique (Anti-Stock Négatif)     | Interface : Tenter de vendre 9999 unités d'un article en stock limité (ex: 5 unités).<br>Preuve SQL : Constaté le message toast rouge d'erreur renvoyé par PostgreSQL : "Stock insuffisant pour ce produit". Zéro corruption, vente annulée.                                                                                                    |
| Scénario E : Coupure Réseau & Synchronisation Automatique              | Interface : Couper le serveur Node.js. Le badge passe en Ø=à Hors-Ligne. Effectuer un TK-OFF-XXXX émis. Relancer le serveur -> Synchronisation réussie.<br>Preuve SQL : La vente est insérée dans `ventes` avec le flag synchronisé et le stock décrémenté rétroactivement.                                                                     |

RÉPONSES TYPES AUX QUESTIONS TECHNIQUES & ARCHITECTURALES DU JURY

| Question Piège du Jury                                                                                          | Réponse Technique & Argumentaire Recommandé                                                                                                                                                                                                                                                                                                                                                                                                             |
|-----------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pourquoi avoir mis la logique dans PostgreSQL plutôt que dans Node.js / Express ?                               | Pour respecter les garanties ACID (Atomicité, Cohérence, Isolation, Durabilité). Si le serveur web crashe ou redémarre pendant une vente, une transaction gérée par Node.js pourrait laisser des données incohérentes (ex: argent encaissé mais stock non débité). Avec une procédure stockée PL/pgSQL encapsulée dans un bloc transactionnel, c'est le moteur PostgreSQL qui garantit que tout réussit ou que tout est annulé (ROLLBACK).              |
| Comment le système gère-t-il la concurrence si 2 caisses vendent le dernier article à la même milliseconde ?    | PostgreSQL utilise le mécanisme de verrouillage de ligne (Row-Level Locking). Lors de l'exécution de <code>`effectuer_vente`</code> , la clause <code>`UPDATE produits SET stock_actuel = stock_actuel - quantité`</code> pose un verrou exclusif sur la ligne de l'article. La première transaction passe, tandis que la seconde déclenche l'exception du trigger <code>`trg_empecher_stock_negatif`</code> et annule la vente concurrente proprement. |
| Comment garantisiez-vous que le mode hors-ligne ne crée pas de doublons lors de la reconnexion ?                | Chaque vente générée hors-ligne reçoit un identifiant UUID unique local et une clé d'idempotence stockée dans IndexedDB. Lors de la synchronisation, le backend vérifie l'existence préalable de cet identifiant avant toute insertion, éliminant tout risque de double facturation.                                                                                                                                                                    |
| Quelle est la différence entre une Procédure Stockée (PROCEDURE) et une Fonction (FUNCTION) dans votre projet ? | Une FUNCTION (04_fonctions.sql) retourne obligatoirement une valeur ou une table et ne peut pas exécuter d'instructions de contrôle de transaction (COMMIT/ROLLBACK) autonomes. Une PROCEDURE (05_procedures.sql), introduite en PostgreSQL 11+, est appelée via CALL, peut manipuler des paramètres INOUT et gérer des transactions complètes.                                                                                                         |
| Comment assurez-vous la conformité avec la réglementation comptable OHADA et les spécificités d'Abidjan ?       | 1. Monnaie officielle : Franc CFA (XOF / FCFA) sans centimes fractionnaires sur les espèces.<br>2. Intégration native des paiements Mobile Money (Wave, Orange Money, MTN) omniprésents en Côte d'Ivoire.<br>3. Piste d'audit immuable (Audit Trail) dans <code>`mouvements_stock`</code> interdisant toute modification rétroactive.                                                                                                                   |